

A Network Analysis of Wikipedia

Joshua Beard

July 2026

The game of trying to connect two Wikipedia pages by clicking on links has probably been played by most nerdy high school students (at least that I know). This game relies on the fact that (almost) all Wikipedia pages are linked to by other pages, and those pages in turn by others - it is a sort of "6 degrees of separation" problem. This game motivated me to characterize Wikipedia's network, but not in ways that have already been done.

People have already created graphs from Wikipedia internal links, and just stopping here (or going one step further) does not interest me. I want to really understand the higher order features of this network. My questions follow from this philosophy.

1 The Questions

1.1 How far apart are two articles, on average?

As a precursor to all these questions, I will have read in Wikipedia as a graph, where nodes are articles and edges are links between articles. Wikipedia publishes their entire website as a giant (10 gb) xml file. I am parsing this xml file with python's pre-packaged xml parser: `xml.etree.ElementTree`. This class has a function, `parse`, which takes an xml file and returns a recursively iterable tree where nodes are tags in the xml file. Wikipedia organized the xml file very well, so this part is simple.

Converting this to a graph is not so simple, and is generally speaking going to be a very computationally painful process. I will probably save the graph in another file, so that I do not need to compute it each time I run. Loosely speaking, I intend on using `mwparserfromhell` to parse the actual wikipedia text, which has a special format for internal links (`[[other wiki article]]`). Once this graph is created, I am prepared to answer the questions.

To solve this first question, I will implement the A* pathfinding algorithm (which I have done before, so does not dramatically increase scope), and randomly pick articles from the graph to find a path between. I will do this for as long as it takes for the expected error to be below a certain threshold, which can be computed statistically. The interest of solving this problem is it tells us what the degree of separation between two words, concepts, or ideas in general (as represented by a wikipedia article) are for the human knowledge-base. I feel like there is something a little philosophically interesting about knowing what this average path length is. I imagine it will be quite short.

As a side note, I recognize that for wikipedia, where there is no well-formed concept of a naive distance between nodes, A* is no more efficient than a BFS. Regardless, implementing it this way allows me to add a naive distance in the future if I come up with a way to represent this idea, perhaps using some level of embedding for wikipedia titles. It would be interesting to pursue in the future, but will not be looked at for my project.

1.2 What is the clustering coefficient of Wikipedia?

The clustering coefficient tells you how much a graph clusters within regions, as opposed to being more homogeneously arranged. As the wikipedia article on the clustering coefficient mentions, social networks tend to be quite clustered. I am curious how clustered wikipedia is. I will probably go for the global clustering coefficient, if it is not too computationally hard. If it is too hard, I will look at interesting regions. The method for computing it is simple, and is described in the linked article. If it proves to be too computationally hard, then I will instead find the local clustering coefficient of interesting articles, such as the "Math" page.

Similar to the average shortest path, I think that this gives a rather interesting philosophical insight on how wikipedia arranges our knowledge. A low clustering coefficient would suggest that on average, any article is similarly likely to link to some other article. A high one would suggest that certain articles (probably near each other), are more likely to link to each other. An interesting extension of this part would be to find the local clustering coefficient for all wikipedia articles, save them, and use them to improve pathfinding in section 1.1 since articles with a high coefficient are more likely to have more connections.

1.3 Page rank Wikipedia

This one is a doozy. After doing some research, doing a full page rank on all wikipedia articles would require diagonalizing a $8 \cdot 10^6 \times 8 \cdot 10^6$ matrix. This is clearly infeasible, but the mathematically pure way is not how google originally did it either! It was done using the following interesting process.

The idea of page rank is given n websites, you build an $n \times n$ matrix (lets call it A) where each entry A_{ij} represents the chance a random walk around the web (clicking random links) takes you from website i to website j . This, of course, depends on how many links each website has on it. After constructing this matrix, we recognize that (by construction) multiplying this matrix by a vector of length n given by:

$$r = \begin{bmatrix} n^{-1} \\ n^{-1} \\ \vdots \\ n^{-1} \end{bmatrix}$$

gives us the probability of being on each node r_i after starting in a random node and taking one step. Each subsequent multiplication by this matrix on this vector gives this probability for additional steps. It is easy to see then, that what we want to rank our pages by how well connected they are, is to find:

$$r_\infty = A^\infty r$$

The easiest way to do this would be to diagonalize A as PDP^{-1} , and use the property that:

$$(PDP^{-1})^n = PD^nP^{-1}$$

which just lets us raise a diagonal matrix to this power, which is simple. Sadly for us, my laptop can not possibly hope to diagonalize A for all of wikipedia, even if its a relatively sparse matrix. As such, we have to use google's statistical approach (sadly leaving the pretty math behind).

The solution they found is motivated by the following issue. I think it is easy to see that you can find approximately r_∞ by just taking $A^n r$ for larger and larger n , until this more or less converges. Sadly, this only works if the graph for forms A is strongly connected and aperiodic (basically you can get to all websites with links, and websites don't link back and forth to each other in certain ways that causes $A^n r$ to oscillate every time n increases). Clearly our idea won't work because the internet is not like this, so here is what google did.

Instead of $r_{n+1} = Ar$, they do:

$$r_{n+1} = \alpha Ar_n + (1 - \alpha)r_0$$

For google, $\alpha = 0.85$ was historically used, so that is what we will do.

In words, this just means that some of the time (set by α), you move between websites, and the other times $(1 - \alpha)$, you jump to a random website using a uniform distribution (which is what r_0 happened to be, so its notationally nice to use it). If we do this, the probability vector r converges to the correct page rank for a much wider range of graphs. All we will need to do is perform these steps iteratively.

Now doing these multiplications is hard too, and requires more ram than my laptop has, so the matrix needs to be represented sparsely (since it is mostly zeros). I will do this with `csr_matrix` from `scipy.sparse`. This matrix implementation lets us represent our data without using space for zeros, and do fast multiplication without killing our ram.

2 Challenge Goals

I will be doing the messy data and new library challenge goals. The data is an xml format that is not particularly conducive to creating this graph. It will require a lot of pre-processing to get the data into a usable state. The new libraries are `mwparsersfromhell` (linked in section 1.1) and `scipy.sparse`. I may end up needing more libraries, but at a minimum I will use these.

3 Work Plan

1. Process XML into graph, and store it in a separate file so I do not need to process the XML each time. This will probably take about 3 hours.
2. Run A* (or in our case, basically a BFS) on this graph between random pairs of nodes, updating the average distance as we go. Historical average distances should be stored in a list where the index is the iteration number. The reason

for this is we can plot iterations versus the average to qualitatively confirm convergence. This will probably take about 3-4 hours.

3. Write and execute the algorithm for clustering coefficient. I do not imagine this will be too hard, since you only need to loop over each node once and find its local properties to compute the global result. This will probably take about 2 hours.
4. Do the page ranking process. First, I will compute the probability adjacency matrix A , and store it as a sparse matrix using the `scipy.sparse` library. Next, I will build the r vector (which is trivial) and run the algorithm on these two objects until the results are sufficiently stable (a threshold which is TBD). This will probably take about 5-6 hours, depending on how it goes.

The total time spent will be about 15 hours at most.